BABA LAGIN

API Flow Diagrams (UI -> DB)

End-to-end request flow for each API across all layers,
with the actual source file names

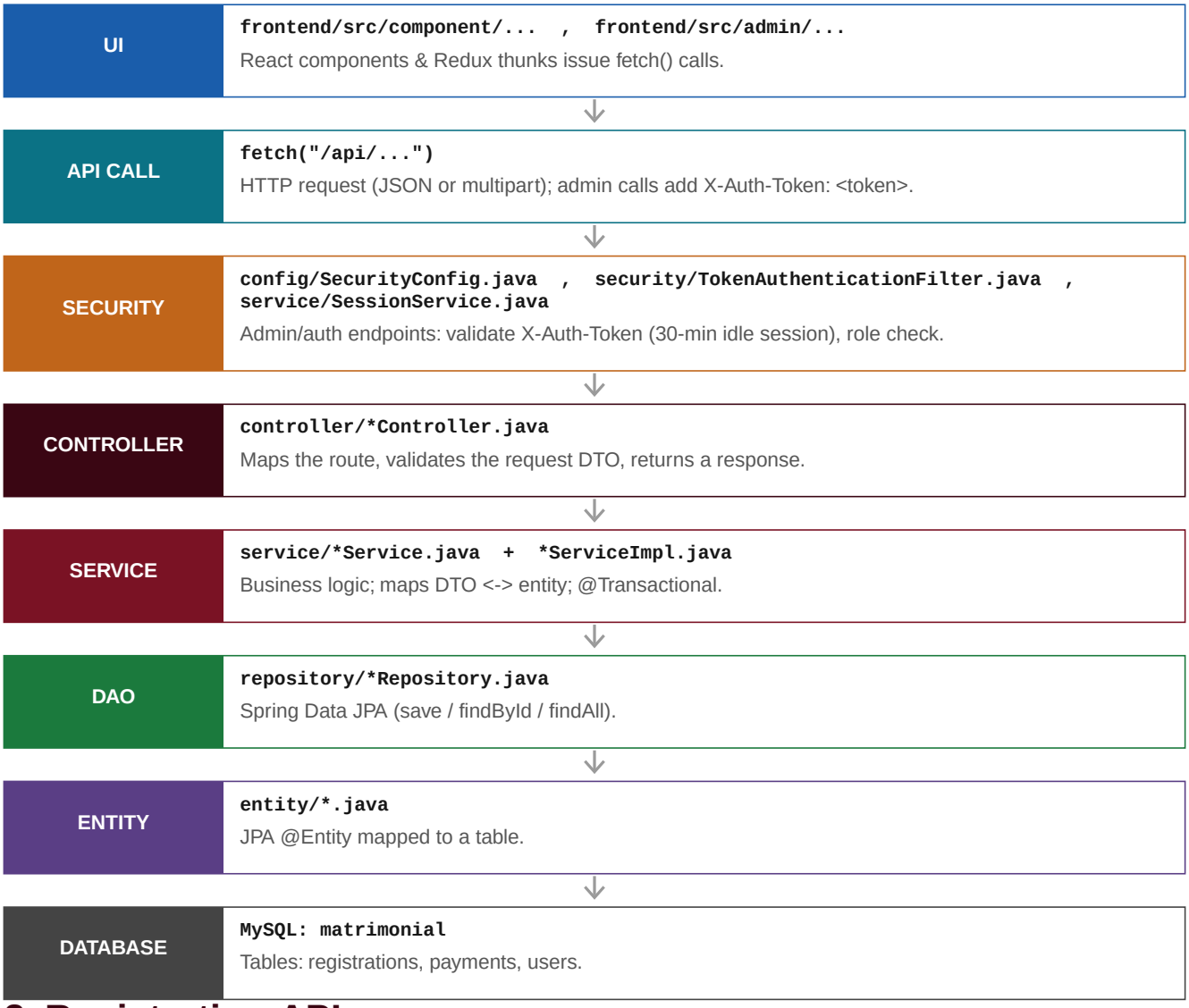
Version	1.0
Date	June 27, 2026
Application	loginapp (Spring Boot 2.7.18 + React)
Layers	UI -> API call -> [Security] -> Controller -> Service -> DAO -> Entity -> DB

1. Architecture & Layers

Every request flows through the same layered stack. The React SPA (built into the Spring Boot static resources) calls a REST endpoint; Spring Security checks access (for admin/auth endpoints); a Controller receives it, delegates to a Service, which uses a Spring Data Repository (DAO) to read/write JPA Entities mapped to MySQL tables. The diagrams that follow show this chain per API with the real file names.

Source locations

Frontend: frontend/src/ | Backend: src/main/java/com/example/login/ | DB config: src/main/resources/application.properties



2. Registration APIs

POST /api/registration

Access: Public

UI	**component/NavbarMenus/Registration.jsx** Enroll form. handleSubmit() -> PaymentDialog -> handlePaymentSubmit().
API CALL	**Slices/registrationSlice.jsx » submitRegistration** fetch POST /api/registration with JSON formData (6 sections).
CONTROLLER	**controller/RegistrationController.java » create()** **dto/RegistrationRequest.java , dto/RegistrationResponse.java** @Valid @RequestBody (email required) -> 201 { id, registrationCode, message }.
SERVICE	**service/RegistrationService.java** **service/RegistrationServiceImpl.java » register()** Maps DTO to entity; generates registrationCode (REG00001) after save.
DAO	**repository/RegistrationRepository.java » save()** Spring Data JpaRepository.
ENTITY	**entity/Registration.java** **entity/{Personal,Horoscope,Education,Address,Family,Expectation}.java** Embedded section value objects.
DATABASE	**MySQL table: registrations** One row per submission.

GET /api/registration (and /{id})

Access: Admin (X-Auth-Token)

UI / CLIENT	**(admin / integration client)** Reads registrations (no dedicated UI tab yet).
API CALL	**fetch GET /api/registration (X-Auth-Token: <token>)** List all, or by id.
SECURITY	**config/SecurityConfig.java** GET /api/registration** requires hasRole('ADMIN').
CONTROLLER	**controller/RegistrationController.java » getAll() / getOne()** by-id 404 via exception/GlobalExceptionHandler.java.
SERVICE	**service/RegistrationServiceImpl.java » getAll() / getById()**

DAO	`repository/RegistrationRepository.java » findAll()/findById()`
DATABASE	`MySQL table: registrations`

POST /api/login (profile login)

Access: Public

UI	`component/Login/Login.jsx » handleSubmit()` Profile Login form: Registration ID + email (+ captcha).

API CALL	`fetch POST /api/login`
	`body { registrationId, email }`
	JSON request.

CONTROLLER	`controller/LoginController.java » login()`
	`dto/LoginRequest.java , dto/LoginResponse.java`
	200 on match+verified (returns a session token); 401 bad creds; 403 payment not verified.

SERVICE	`service/RegistrationServiceImpl.java » login()`
	`service/SessionService.java » create()`
	Looks up by registrationCode + email; issues a 30-min idle session token.

DAO	`repository/RegistrationRepository.java`
	`» findByRegistrationCodeAndEmail()`
	JPQL, case-insensitive email.

PAYMENT GATE	`service/PaymentServiceImpl.java » getByRegistrationId()`
	`repository/PaymentRepository.java » findByRegistrationId()`
	Login allowed only if a payment for this code is VERIFIED; else 403 (pending/rejected).

DATABASE	`MySQL tables: registrations + payments` Validates registration_code + p_email, and payment status = VERIFIED.

GET /api/profiles/search (Find Your Life Partner)

Access: Public

UI	`component/NavbarMenus/Carousel.jsx » handleSearch()`
	`component/NavbarMenus/MatchinSearch.jsx » handleSubmit()`
	Hero form OR navbar Search > Matching Search -> /search/results.

UI	`component/NavbarMenus/SearchResults.jsx` Reads query params, renders profile cards.
API CALL	`fetch GET /api/profiles/search?lookingFor=&ageFrom=&...`

CONTROLLER	**controller/ProfileSearchController.java » search()** **dto/ProfileSummary.java** Maps query params to filters.
SERVICE	**service/RegistrationServiceImpl.java » search()** Age range -> birth-year range; ignores blank/Any.
DAO	**repository/RegistrationRepository.java » search()** JPQL with optional filters (gender, maritalStatus, edu, city, community, year).
DATABASE	**MySQL table: registrations** Matches p_gender, p_community, city, education, dob_year.

3. Payment APIs

POST /api/payment/verify

Access: Public (multipart/form-data)

UI	**component/NavbarMenus/Registration.jsx (PaymentDialog)** handleSubmitPayment(): transactionId + screenshot file.
API CALL	**Slices/registrationSlice.jsx » submitPayment** fetch POST /api/payment/verify with FormData (multipart).
CONTROLLER	**controller/PaymentController.java » verify()** **dto/PaymentResponse.java** @RequestParam fields + MultipartFile -> 201.
SERVICE	**service/PaymentService.java** **service/PaymentServiceImpl.java » verifyPayment()** Stores screenshot to disk, then saves record.
DAO	**repository/PaymentRepository.java » save()**
ENTITY	**entity/Payment.java** status = PENDING_VERIFICATION.
DATABASE	**MySQL table: payments** **File: uploads/payments/<uuid>.<ext>** Row + screenshot file on disk.

PUT /api/payment/{id}/status

Access: Admin (X-Auth-Token, ROLE_ADMIN)

UI

`admin/AdminDashboard.jsx » updateStatus()`
`admin/adminAuth.js (X-Auth-Token header)`
Verify / Reject buttons (with optional note).

↓

API CALL

`fetch PUT /api/payment/{id}/status`
`body { status, note } + X-Auth-Token: <token>`

↓

SECURITY

`config/SecurityConfig.java`
`service/CustomUserDetailsService.java`
PUT .../status requires ROLE_ADMIN; user validated from DB.

↓

CONTROLLER

`controller/PaymentController.java » updateStatus()`
`dto/PaymentStatusUpdateRequest.java`
Validates status in {VERIFIED,REJECTED,PENDING_VERIFICATION}.

↓

SERVICE

`service/PaymentServiceImpl.java » updateStatus()`
Sets status, note, updatedAt.

↓

DAO

`repository/PaymentRepository.java » findById() + save()`

↓

DATABASE

`MySQL table: payments (entity/Payment.java)`

GET /api/payment (and /{id})

Access: Admin (X-Auth-Token, ROLE_ADMIN)

UI

`admin/AdminDashboard.jsx » loadPayments()`
`admin/adminAuth.js`
Loads the payments table on the dashboard.

↓

API CALL

`fetch GET /api/payment (X-Auth-Token: <token>)`

↓

SECURITY

`config/SecurityConfig.java`
GET /api/payment** requires ROLE_ADMIN.

↓

CONTROLLER

`controller/PaymentController.java » getAll() / getOne()`

↓

SERVICE

`service/PaymentServiceImpl.java » getAll() / getById()`

↓

DAO

`repository/PaymentRepository.java » findAll()/findById()`

↓

DATABASE	`MySQL table: payments`

GET /api/payment/{id}/screenshot

Access: Admin (X-Auth-Token, ROLE_ADMIN)

UI	`admin/AdminDashboard.jsx » viewScreenshot()` Fetches blob with X-Auth-Token header, opens in a new tab.
API CALL	`fetch GET /api/payment/{id}/screenshot`
SECURITY	`config/SecurityConfig.java` ROLE_ADMIN required.
CONTROLLER	`controller/PaymentController.java » screenshot()` Returns the image as a UrlResource.
SERVICE	`service/PaymentServiceImpl.java » getById()` Reads screenshot_path from the record.
DAO	`repository/PaymentRepository.java » findById()`

SOURCE	`Disk: uploads/payments/<file>`
	`(path from payments.screenshot_path)`
	Streams the stored image bytes.

4. Documents (Profile Photo) API

POST /api/documents/upload

Access: Public (multipart/form-data)

UI	`component/Navbar.jsx » handleProfileUpload()` Click profile photo box / name -> pick image file.
API CALL	`fetch POST /api/documents/upload (FormData)` registrationCode + docType=profileImage + file.

CONTROLLER	`controller/DocumentController.java » upload()`
	`dto/DocumentResponse.java`
	Returns saved doc as a data URL.

SERVICE	`service/DocumentService.java`
	`service/DocumentServiceImpl.java » upload()`
	Base64-encodes the file bytes.

DAO	`repository/DocumentRepository.java » save()`
ENTITY	`entity/Document.java` @Lob base64 column; older rows retained.
DATABASE	`MySQL table: documents` Latest profileImage = active photo.

GET /api/documents/profile-image

Access: Public

UI	`component/Navbar.jsx (useEffect on login)` Loads the photo box; dummy avatar if none.
API CALL	`fetch GET /api/documents/profile-image?registrationCode=`
CONTROLLER	`controller/DocumentController.java » profileImage()` 200 data URL, or 404 if none.
SERVICE	`service/DocumentServiceImpl.java » getLatestByType()`

DAO	`repository/DocumentRepository.java`
	`» findFirstByRegistrationCodeAndDocTypeOrderByIdDesc()`
	Latest profileImage row.

DATABASE	`MySQL table: documents`

GET /api/admin/profiles (Admin > Profiles)

Access: Admin (X-Auth-Token, ROLE_ADMIN)

UI	`admin/AdminLayout.jsx (navbar)`
	`admin/AdminProfiles.jsx`
	Filters: isActive, created date range, payment status.

API CALL	`fetch GET /api/admin/profiles?... (X-Auth-Token: <token>)`
SECURITY	`config/SecurityConfig.java` /api/admin/** requires ROLE_ADMIN.

CONTROLLER	`controller/AdminController.java » profiles()`
	`dto/AdminProfileView.java`
	Joins each registration with its latest payment status.

SERVICE	`service/RegistrationServiceImpl.java » adminSearch()` `service/PaymentServiceImpl.java » getByRegistrationId()`

DAO	`repository/RegistrationRepository.java » adminSearch()`
	`repository/PaymentRepository.java`
	isActive + createdDate range filter.

DATABASE	`MySQL tables: registrations + payments` All registration columns + payment status.

5. Contact Us API

POST /api/contact (public submit)

Access: Public

UI	`component/ContactUs.jsx » handleSubmit()` Contact form: name, phone, email, subject, message.
API CALL	`fetch POST /api/contact (JSON)`

CONTROLLER	`controller/ContactController.java » submit()`
	`dto/ContactRequest.java`
	@Valid -> 201 { id, message }.

SERVICE	`service/ContactService.java » save()`
DAO	`repository/ContactMessageRepository.java » save()`
DATABASE	`MySQL table: contact_messages (entity/ContactMessage.java)`

GET /api/admin/contacts (Admin > Contact Messages)

Access: Admin (X-Auth-Token, ROLE_ADMIN)

UI	`admin/AdminContacts.jsx` Table of all contact submissions.
API CALL	`fetch GET /api/admin/contacts (X-Auth-Token: <token>)`
SECURITY	`config/SecurityConfig.java` /api/admin/** requires ROLE_ADMIN.
CONTROLLER	`controller/AdminController.java » contacts()`

SERVICE	`service/ContactService.java » getAll()`
DATABASE	`MySQL table: contact_messages` Newest first.

6. Notifications & Interest API

POST /api/interest (express interest)

Access: Logged-in profile user

UI	`component/Profile/ProfileListPage.jsx » handleInterest()` Express Interest button (card or View Profile popup).
API CALL	`fetch POST /api/interest { fromCode, toCode }`
CONTROLLER	`controller/NotificationController.java » express()`
SERVICE	`service/NotificationService.java » expressInterest()` Creates INTEREST_RECEIVED for the target.
DAO	`repository/NotificationRepository.java » save()`
DATABASE	`MySQL table: notifications (entity/Notification.java)`

GET /api/notifications (navbar bell)

Access: Logged-in profile user

UI	`component/Navbar.jsx (bell + count)`
	`component/NavbarMenus/NotificationPanel.jsx`
	Bell shown only after login; panel marks read on open; Accept/Reject on interests.

API CALL	`GET /api/notifications?registrationCode=`
	`POST /api/notifications/read`
	`POST /api/interest/respond`

CONTROLLER	`controller/NotificationController.java` list / markRead / respond.
SERVICE	`service/NotificationService.java` respondToInterest -> INTEREST_ACCEPTED/REJECTED to sender.

DATABASE	**MySQL table: notifications** Also created on admin payment status changes.

7. Authentication API

POST /api/admin/login (admin login)

Access: Public

UI	**admin/AdminLogin.jsx » handleSubmit()** Username + password; requires ROLE_ADMIN to proceed.
↓	
API CALL	**fetch POST /api/admin/login** **body { username, password }** On 200, stores token + instanceId via admin/adminAuth.js (saveAuth).
↓	
CONTROLLER	**controller/AdminAuthController.java » login()** 200 { token, username, roles, instanceId }; 401 bad creds; 403 not admin.
↓	
SECURITY	**repository/AppUserRepository.java** **BCrypt PasswordEncoder** Loads user from DB; verifies password; checks role ADMIN.
↓	
SERVICE	**service/SessionService.java » create()** Issues an in-memory session token with a 30-min idle timeout.
↓	
DATABASE	**MySQL table: users (entity/AppUser.java)** **Seeded by config/DataInitializer.java** Admin account read during authentication.

GET/POST /api/session/validate & /api/session/heartbeat

Access: Token (X-Auth-Token)

UI	**component/SessionValidator.jsx (profile)** **admin/RequireAdmin.jsx (admin)** Polls validate (every 60s) and sends heartbeat on user activity; logs out on 401.
↓	
API CALL	**fetch GET /api/session/validate (X-Auth-Token: <token>)** **fetch POST /api/session/heartbeat (X-Auth-Token: <token>)** No body; token in header.
↓	
SECURITY	**security/TokenAuthenticationFilter.java** Reads X-Auth-Token; authenticates if the session is still live.
↓	

CONTROLLER	**controller/SessionController.java » validate() / heartbeat()** validate: 200/401 (read-only). heartbeat: extends window, 200/401.
SERVICE	**service/SessionService.java » isValid() / touch()** In-memory store; 30-min idle expiry. validate does NOT extend; heartbeat does.
STATE	**(in-memory ConcurrentHashMap, no DB table)** Sessions are wiped on restart, so a restart also logs everyone out.

8. Messaging API

POST / GET /api/messages (chat between accepted connections)

Access: Logged-in profile user

UI	**component/ChatModal.jsx** **opened from Profile/ProfileListPage.jsx & NavbarMenus/SingleIdSearch.jsx** Input box on top; history below (polls every 5s). Shown only when accepted.
API CALL	**POST /api/messages { fromCode, toCode, content }** **GET /api/messages?code=&withCode=** Send a message / load conversation.
CONTROLLER	**controller/MessageController.java » send() / conversation()** 403 unless the two are accepted-connected; 400 on empty content.
SERVICE	**service/MessageService.java** **service/NotificationService.java » areConnected()** Gate on accepted interest; also creates a MESSAGE notification for the recipient.
DAO	**repository/MessageRepository.java » findConversation()** Both-directions conversation, oldest first.
DATABASE	**MySQL table: messages (entity/Message.java)** Email/mobile are never exposed; connected users talk here instead.

9. Profile Renewal

POST /api/renewal/validate + /api/payment/verify (RENEWAL)

Access: Public

UI

component/Login/RenewalPage.jsx
component/Login/Renewal.jsx (form)
reuses NavbarMenus/Registration.jsx » PaymentDialog

Renew menu: id+email+period+captcha -> validate -> payment dialog (period price).

↓

API CALL

POST /api/renewal/validate { registrationId, email }
POST /api/payment/verify (multipart, paymentType=RENEWAL, renewalMonths)

Validate the profile, then submit the renewal payment.

↓

CONTROLLER

controller/RenewalController.java » validate()
controller/PaymentController.java » verify()

404 if id+email do not match; payment saved PENDING_VERIFICATION.

↓

SERVICE

service/PaymentServiceImpl.java » verifyPayment() / updateStatus()

On admin VERIFIED of a RENEWAL: set isActive=Y and renewed_until = now + months.

↓

DATABASE

MySQL tables: payments (payment_type, renewal_months)
registrations (is_active, renewed_until)

Admin verifies the renewal payment in Payments Admin -> profile reactivated.